Project Title	**Intelligent Document Processing System using NLP, Computer Vision, and Cloud Deployment**
Skills take away From This Project	**Deep Learning, Computer Vision, Optical Character Recognition (OCR) using EasyOCR / Tesseract / TrOCR, Natural Language Processing (NLP), Named Entity Recognition (NER) using SpaCy / HuggingFace, Data Preprocessing and Cleaning, Transformer Model Fine-tuning, API Development using FastAPI / Flask, Frontend Design using Streamlit , Cloud Deployment (AWS), Database Integration, Version Control using Git**
Domain	**Artificial Intelligence / Natural Language Processing / Machine Learning / Document AI / Enterprise Automation**

Problem Statement:

Learners are required to build an **Intelligent Document Processing (IDP) system** that can automatically extract structured information from unstructured documents such as invoices, ID cards, resumes, and reports using OCR, NLP, and Deep Learning techniques, and deploy the solution on the cloud.

Business Use Cases

1. Automated Invoice Processing & Accounts Payable

Scenario:

Finance teams manually process large volumes of invoices, leading to delays, human errors, and high operational costs.

Application:

The Intelligent Document Processing system automatically extracts invoice numbers, vendor details, dates, line items, and total amounts from scanned or digital invoices and converts them into structured data for ERP systems—significantly reducing manual effort and processing time.

2. Banking & KYC Document Verification

Scenario:

Banks receive thousands of customer KYC documents (ID proofs, address proofs) in different formats, making manual verification slow and error-prone.

Application:

The system extracts customer name, date of birth, address, and ID numbers from documents such as Aadhaar, PAN, passports, and utility bills, enabling faster and more accurate KYC verification workflows.

GUVI | HCL
Skill Up. Level Up

3. Resume Screening & Talent Acquisition

Scenario:

HR teams spend excessive time manually reviewing resumes to extract candidate details and shortlist profiles.

Application:

The document processing system automatically parses resumes to extract skills, experience, education, and contact details, helping recruiters quickly screen candidates and build structured talent databases.

4. Insurance Claims Processing

Scenario:

Insurance claims involve multiple documents like claim forms, bills, and reports, which are manually reviewed, causing processing delays.

Application:

The system extracts policy numbers, claim amounts, hospital details, and dates from submitted documents, enabling faster claim validation and reducing turnaround time.

5. Healthcare Records Digitization

Scenario:

Hospitals and clinics maintain large volumes of paper-based medical records that are difficult to search and analyze.

Application:

The system digitizes patient records by extracting patient details, diagnosis, prescriptions, and test results from scanned medical documents, enabling easy access and electronic health record (EHR) integration.

6. Logistics & Supply Chain Documentation

Scenario:

Logistics companies deal with invoices, delivery notes, and bills of lading that require manual data entry.

Application:

The document processing system extracts shipment details, invoice values, dates, and consignee information, improving accuracy in tracking, billing, and reconciliation.

8. Government & Public Sector Digitization

Scenario:

Government departments receive applications and forms in multiple formats that require manual data entry.

Application:

The system automatically digitizes application forms, extracts citizen details, and stores structured data, improving processing speed and reducing paperwork.

Approach:

1. **Model Selection**

   Select suitable OCR and NLP models based on document type and language. Choose EasyOCR / Tesseract / TrOCR for text extraction and Hugging Face Transformer models or SpaCy for Named Entity Recognition. Ensure models support multilingual documents and different layouts (invoices, IDs, resumes, etc.).

2. **Document Ingestion & Input Handling**

   Design a file upload mechanism to accept PDF, JPG, and PNG formats. Validate file size, type, and readability before processing. Ensure multiple document uploads are handled efficiently.

3. **Image Preprocessing & Enhancement**

   Apply image preprocessing techniques such as grayscale conversion, noise removal, deskewing, thresholding, and resizing to improve OCR accuracy. Use OpenCV for image enhancement and layout correction.

4. **OCR Extraction Logic**

   Trigger OCR processing on uploaded documents to extract raw text and bounding box information. Ensure efficient handling of multi-page PDFs and scanned images. Route documents dynamically to the appropriate OCR engine.

5. **Text Cleaning & Normalization**

   Clean extracted text by removing noise, extra spaces, special characters, and formatting issues. Normalize text for consistent NLP processing.

6. **NLP & Entity Extraction Logic**

   Apply Named Entity Recognition models to extract key fields such as Name, Date, Address, Invoice Number, Amount, etc. Combine ML-based extraction with rule-based regex patterns for higher accuracy on structured fields.

7. **Post-processing & Validation**

Validate extracted fields using business rules (date format, numeric checks, mandatory fields). Handle missing values and low-confidence predictions gracefully. Highlight uncertain fields in the UI for manual review.

8. **Streamlit UI/UX Design**

Create a responsive and clean interface with sections for document upload, extracted text preview, structured field display, and status messages. Optionally include features like download results, clear button, and confidence indicators.

9. **Backend API Integration**

Develop REST APIs using FastAPI / Flask to handle document upload, processing, and result retrieval. Ensure modular separation between OCR, NLP, and business logic layers.

GUVI | HCL
Skill Up. Level Up

10. **Database & Storage Integration**

Store uploaded documents in cloud storage (S3 / Azure Blob) and extracted structured data in a database (MongoDB / PostgreSQL). Maintain proper schema design for easy querying and auditing.

11. **Error Handling & Feedback**

Handle model loading failures, unsupported translations, or long text inputs gracefully. Provide appropriate user feedback using Streamlit alerts or text boxes.

12. **Model Deployment**

Deploy the Streamlit app on **Streamlit Cloud**, Hugging Face Spaces, or other platforms. Ensure fast startup and proper handling of API/model latency.

13. **Version Control and Documentation**

Maintain code in a GitHub repository with proper versioning, commit messages, and a README file outlining setup, usage, and language support.

Results:

1. **Functional Web Application**: A fully functional web interface built with Streamlit where users can:

   A working web application that:

   - Accepts document uploads
   - Extracts text accurately using OCR
   - Identifies key fields using NLP
   - Displays structured output to users

   Deployed solution accessible via cloud URL

   Ability to handle multiple document formats

   Improved automation compared to manual processing

2. **Scalable Deployment**: A scalable deployment framework using Hugging Face/AWS services.
3. **Documentation**: Comprehensive documentation outlining setup, deployment, and usage instructions.

GUVI | HCL
Skill Up. Level Up

Project Evaluation metrics:

- You are supposed to write a code in a modular fashion (**in functional blocks**)
- Maintainable: It can be maintained, even as your codebase grows.
- Portable: It works the same in every environment (operating system)
- You have to maintain your code on **GitHub**.(Mandatory)
- You have to keep your **GitHub** repo public so that anyone can check your code.(Mandatory)
- Proper readme file you have to maintain for any project development(Mandatory)
- You should include basic workflow and execution of the entire project in the readme file on **GitHub** (Mandatory)
- Follow the coding standards: https://www.python.org/dev/peps/pep-0008/
- You need to Create a Demo video of your working model and post in **LinkedIn(Mandatory)**

Data Set:

Source:

Kaggle – Invoice Dataset, Resume Dataset, ID Card Dataset
RVL-CDIP Document Classification Dataset
Custom scanned documents (provided by trainers)

Format:

PDF, JPG, PNG

Variables:

Text content
Bounding boxes
Labels (Name, Date, Amount, Address, etc.)

Project Deliverables:

1. **Data Files**:

- Data files used in the project - app.py, requirements.txt
- If data cannot be shared, provide information on how to access the data you have used.

2. **Source Code**:

- All scripts and code files used in the project.
- A file or Jupyter Notebook that shows your project in action.
- Ensure the code is well-organized and properly commented for clarity and maintainability.

3. **Documentation - README File:**

- The name of your project and what it's about.
- How to set up everything and run your code.
- A quick overview of how your project is organized.
- Any tools or libraries needed to run your code.

Project Guidelines:

1. Coding Standards

- **Readability**: Write clean and readable code. Use meaningful variable and function names.
- **Comments**: Add comments to explain complex logic or important sections of your code.
- **Consistency**: Stick to a consistent coding style. Use tools like linters (e.g., PEP8 for Python) to maintain consistency.
- **Modularity**: Break your code into functions and modules to make it reusable and easier to understand.
- **Error Handling**: Implement proper error handling to make your code robust and user-friendly.
- **Documentation**: Document your functions and classes using docstrings. Include information about inputs, outputs, and any exceptions.

2. Version Control

- **Use Git**: Use Git for version control to keep track of changes and collaborate with others.
- **Regular Commits**: Commit your code regularly with meaningful commit messages. This helps track progress and rollback changes if needed.
- **Branching**: Use branches for new features or bug fixes. This keeps the main branch stable.
- **Pull Requests**: Use pull requests for code reviews before merging branches. This ensures code quality and catches potential issues early.
- **.gitignore**: Use a .gitignore file to exclude unnecessary files from the repository (e.g., compiled files, temporary files, environment-specific configurations).

Timeline:

14 days from the date of document issuance.

PROJECT DOUBT CLARIFICATION SESSION (PROJECT AND CLASS DOUBTS)

About Session: The Project Doubt Clarification Session is a helpful resource for resolving questions and concerns about projects and class topics. It provides support in understanding project requirements, addressing code issues, and clarifying class concepts. The session aims to enhance comprehension and provide guidance to overcome challenges effectively.
Note: Book the slot at least before 12:00 Pm on the same day

Timing: Monday-Saturday (4:30PM to 5:30PM)

Booking link :https://forms.gle/XC553oSbMJ2Gcfug9

LIVE EVALUATION SESSION (CAPSTONE AND FINAL PROJECT)

About Session: The Live Evaluation Session for Capstone and Final Projects allows participants to showcase their projects and receive real-time feedback for improvement. It assesses project quality and provides an opportunity for discussion and evaluation.
Note: This form will Open only on Saturday (after 2 PM) and Sunday on Every Week

Timing:

For DS and AIML
Monday-Saturday (05:30PM to 07:00PM)

Booking link : https://forms.gle/1m2Gsro41fLtZurRA


GUVI
HCL
Skill Up. Level Up

GUVI
HCL
Skill Up. Level Up